Cut Your Losses in Large-Vocabulary Language Models

Wijmans, Huval, Hertzberg, Koltun, Krähenbühl (Apple) · ICLR 2025

Разбор и воспроизведение на CPU

Карина Денисова · Efficient Models, ИТМО 2026 · ДЗ-2

Проблема: логиты тяжелее самой модели

- Последний слой: логиты = $E \cdot C^T$, размер N × V
- Словари растут: Llama 3 — 128K, Gemma 2 — 256K
- Gemma 2 (2B), N = 8192: одна матрица логитов в fp32 = **8,4 ГБ**
- Обычный CE хранит несколько копий: логиты, softmax, градиент
- В статье: лосс Gemma 2 занимает **24 ГБ**, больше всей остальной модели

E N × D (8192 × 2304)
C V × D (256 000 × 2304)

логиты N × V
= 2,1 млрд чисел

А нужен из них для лосса, по сути, один логит на токен и одна сумма

Идея CCE: не записывать логиты в память

$$\ell_i = \log \Sigma_j \exp(C_j \cdot E_i) - C_y_i \cdot E_i$$

Linear-log-sum-exp

Идём по словарю блоками nb × vb, держим бегущий максимум и сумму экспонент (как FlashAttention)

В памяти только один блок логитов

Indexed matmul

Правильный логит — скалярное произведение E_i с одной строкой C

Вся матрица N × V не нужна

Backward, фильтрация и сортировка

- Логиты блока пересчитываются, $S = \exp(\text{логиты} - \text{LSE})$
- $\nabla E \mathrel{+}= (S - Y) \cdot C_{\text{блок}}, \quad \nabla C_{\text{блок}} \mathrel{+}= (S - Y)^T \cdot E$

Gradient filtering

Если все элементы softmax в блоке $< \varepsilon = 2^{-12}$ (точность bf16), блок пропускается

Экономит 2 из 3 матричных умножений блока

Vocabulary sorting

Словарь упорядочивается по среднему логиту

Значимые токены собираются в немногих блоках, остальные блоки можно пропустить

Что заявляют авторы

24 ГБ → 1 МБ

память на лосс, Gemma 2 (2B)

28 ГБ → 1 ГБ

память классификатора при обучении

≈ torch.compile

скорость, при этом точный результат

- Кривые обучения совпадают с обычным CE
- Реализация на Triton, замеры на GPU (A100/H100)

Что я воспроизводила

#	Утверждение статьи	Как проверяю
1	Точно тот же лосс и градиенты	Сравнение с обычным CE в fp32 и fp64
2	Память не зависит от V	tracemalloc, V от 8K до 128K
3	Обучение не страдает	Обучаю маленькую LM двумя способами
4	Фильтрация + сортировка экономят работу	Доля значимых softmax и пропущенных блоков

Реализация с нуля на numpy: baseline CE, indexed matmul, linear-log-sum-exp, блочный backward с фильтрацией и сортировкой

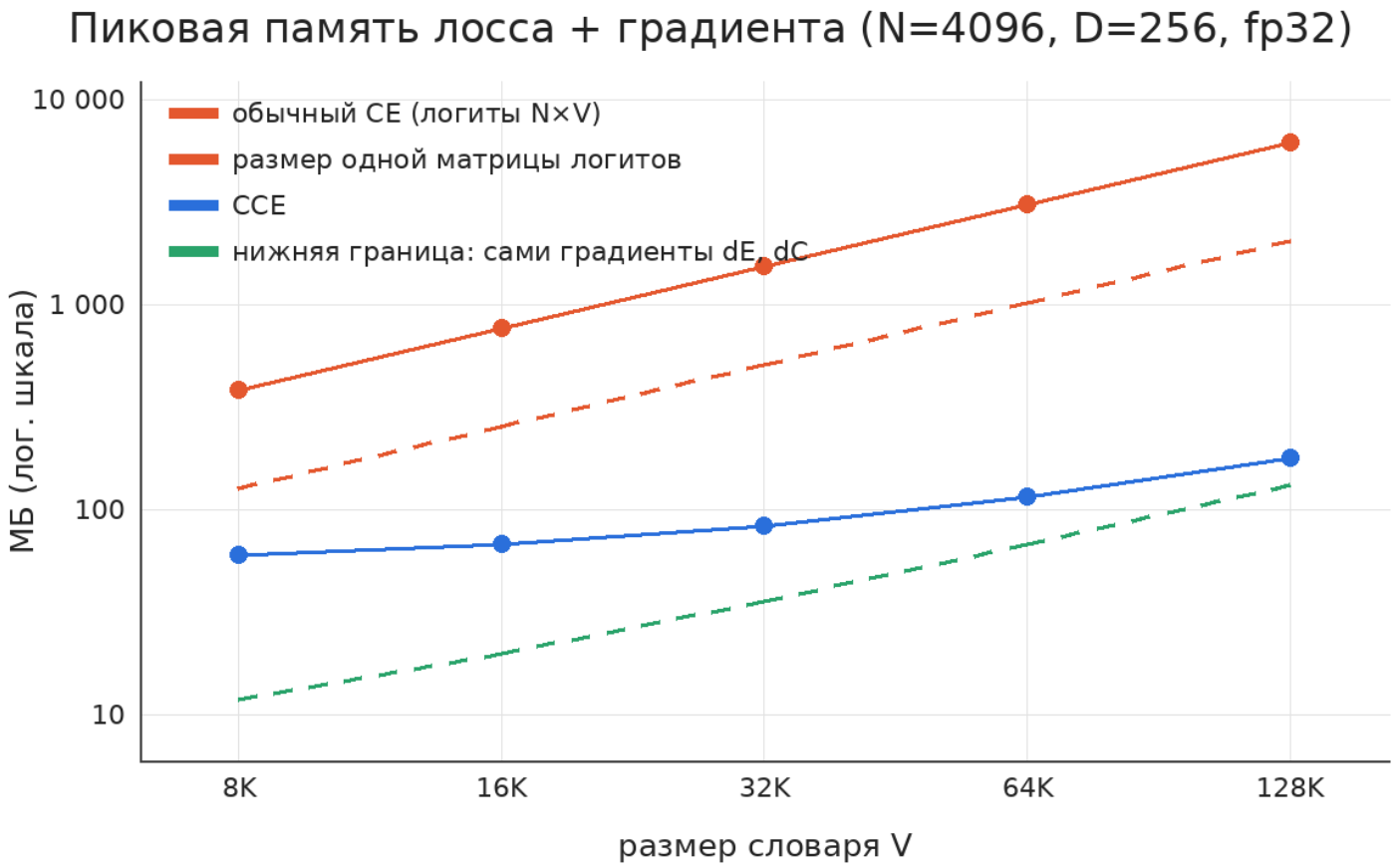
Модель: 4 предыдущих слова → MLP → D = 64, корпус «Opticks» Ньютона, V = 4505. CPU: 4 ядра Xeon

1. Точность: CCE — не приближение

Точность	Вариант	Ошибка лосса	Ошибка ∇E	Ошибка ∇C
fp32	CCE	0	$1{,}0 \cdot 10^{-9}$	$3{,}5 \cdot 10^{-9}$
fp32	CCE + фильтрация + сортировка	0	$1{,}3 \cdot 10^{-9}$	$3{,}5 \cdot 10^{-9}$
fp64	CCE	0	$2{,}2 \cdot 10^{-18}$	$7{,}4 \cdot 10^{-18}$
fp64	CCE + фильтрация + сортировка	0	$2{,}7 \cdot 10^{-18}$	$7{,}4 \cdot 10^{-18}$

N = 1024, D = 64, V = 20 000. Расхождение на уровне машинной точности.
✅ Подтверждено

2. Память: CCE не зависит от словаря



V = 128K, N = 4096

обычный CE: **6 144 МБ**

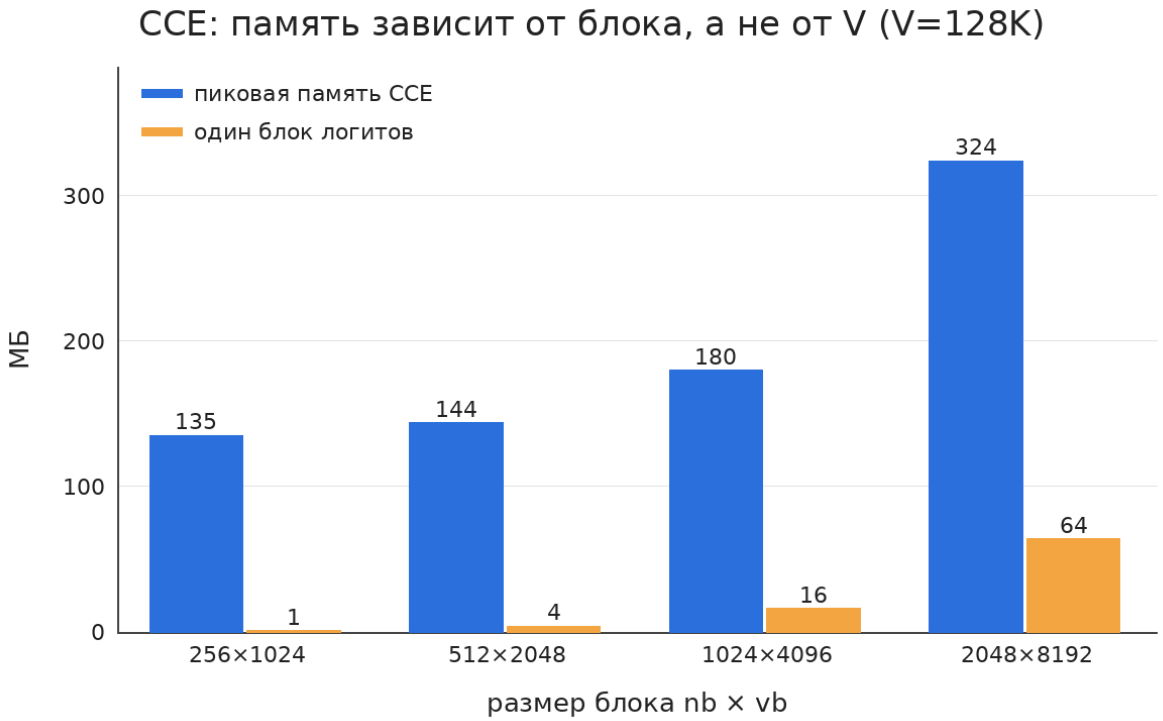
CCE: **180 МБ** **(34×)**

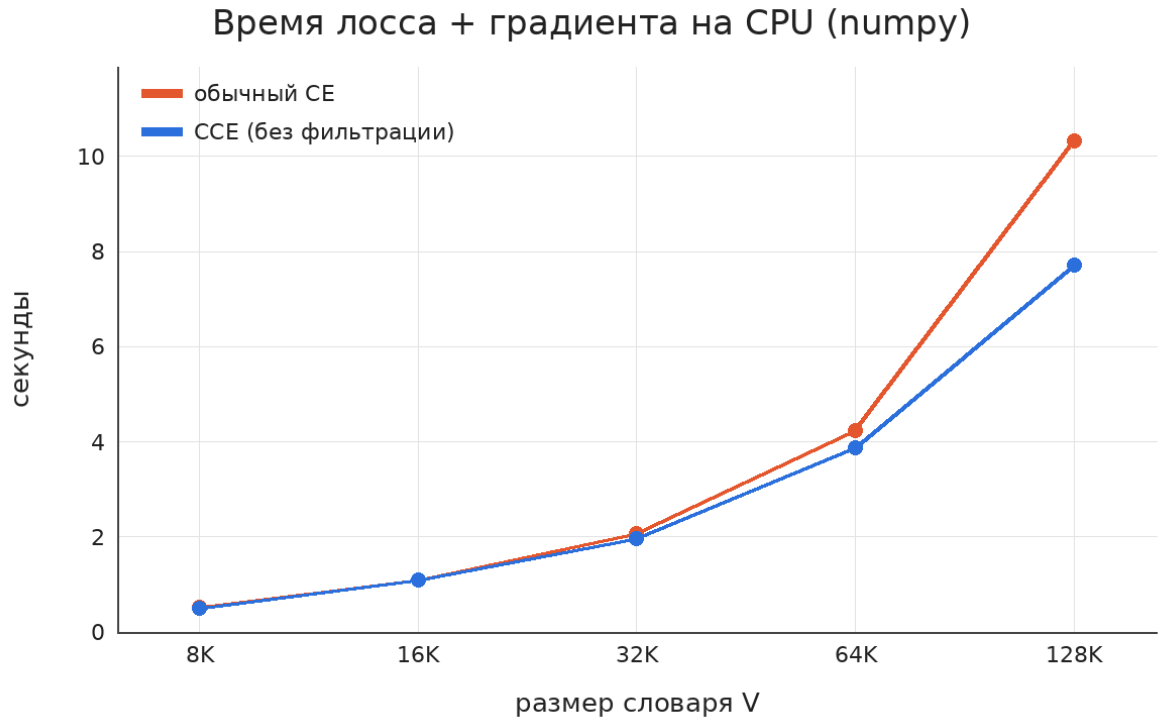
Обычный CE = 3 матрицы логитов

CCE = градиенты (132 МБ) + один блок

Forward CCE: 48 МБ при любом V

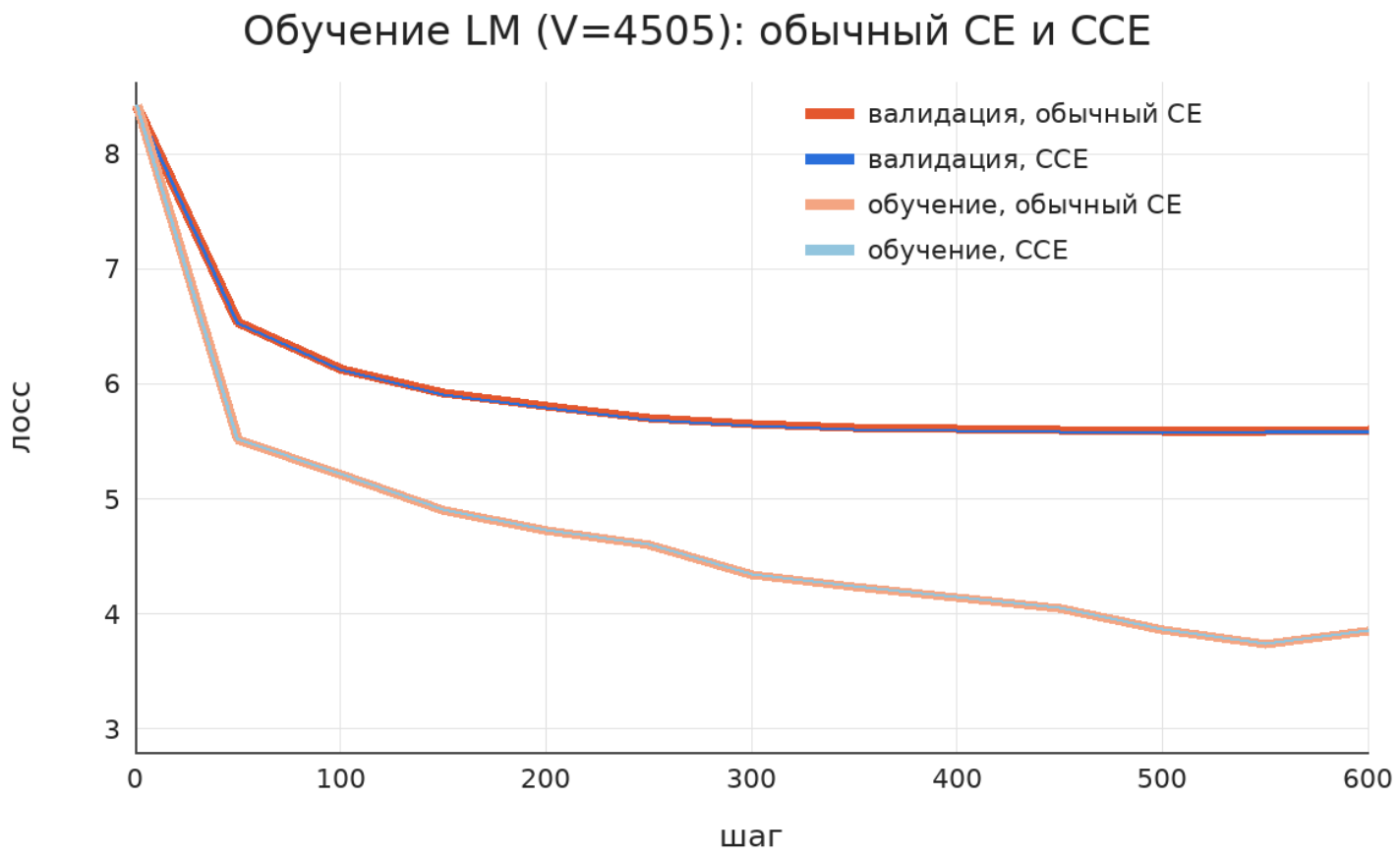
Память задаёт блок, время не хуже





V = 128K: обычный CE 10,3 с, CCE 7,7 с на 4 ядрах CPU. ✅ Подтверждено

3. Обучение: кривые совпадают

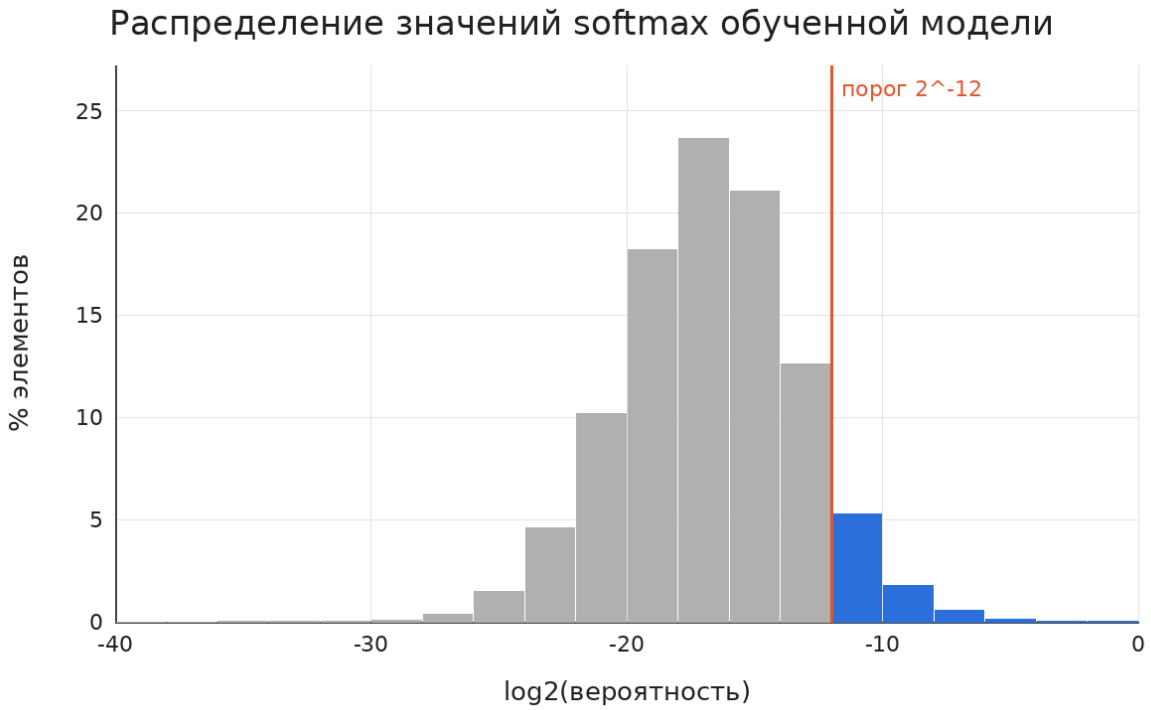


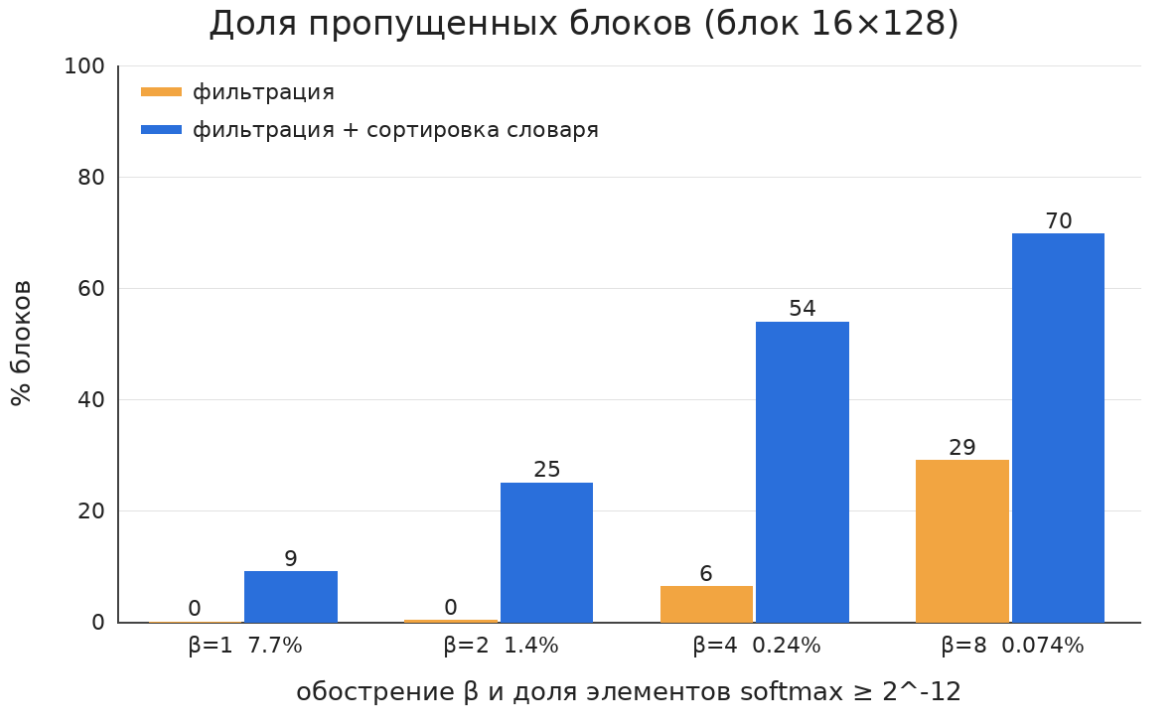
	CE	CCE
train, шаг 600	3.856	3.855
val, шаг 600	5.601	5.587
время, с	45	41

Макс. расхождение val: 0,023

✅ **Подтверждено**

4. Фильтрация и разреженный softmax





Без сортировки при β = 4 пропускается 6% блоков, с сортировкой 54%. Ошибка градиента ~10^{-4}, ниже точности bf16

⚠ Механизм подтверждён, но разреженность настоящей LLM на маленькой модели не проверить

Сверка с цифрами статьи (Gemma 2 2B)

Величина	Формула	Расчёт	В статье
Обычный CE: 3 матрицы логитов, fp32	3 · N · V · 4 Б	23,4 ГиБ	24 ГБ
Forward CCE: только вектор LSE	N · 4 Б	32 КиБ	1 МБ
Backward: ∇C + ∇E в bf16 (нижняя граница)	(V + N) · D · 2 Б	1,13 ГиБ	≈ 1 ГБ

N = 8192, V = 256 000, D = 2304. «3 матрицы логитов» из моих замеров точно объясняет 24 ГБ из статьи

Итоги

Утверждение	Результат
Точно совпадает с обычным CE	✅ до машинной точности
Память не зависит от V	✅ 6 144 → 180 МБ при V = 128K (34×)
Обучение не страдает	✅ кривые совпадают
Фильтрация + сортировка	✅ до 70% блоков пропускается; ⚠ зависит от разреженности
Не медленнее обычного CE	✅ на CPU; GPU не проверялся

Не воспроизведено: скорость Triton-ядер на GPU и разреженность softmax настоящих LLM

Вывод: CCE — бесплатный выигрыш по памяти при большом словаре